DOCUMENT FOR LAB WORK 4

CONTENTS

Problem Description 1
Input for newly added function 2
Output for newly added function 2
Data Structure 3
Algorithm 4
Algorithm: 4
Pseudocode: 4
Test Case 5

PROBLEM DESCRIPTION

Previously, the robot operated in the environment containing buildings and dirt. Its primary task was to clean dirt using random movement.

In Lab 4 task the robot's behavior has been enhanced:

- While moving randomly, if the robot encounters a human, it will divert to an alternative path.
- The alternative path is chosen based on the nearest object to the robot.
- This simulation should be run 100 times to evaluate the behavior.

New requirements:

- We have to write test code for the newly added functionality [`move_to_nearest_object()`]
- In the console, we have to print the following statistics:
  - Number of humans encountered
  - Number of times the nearest path was selected and avoided
  - If there is no human in its current position, the robot continues moving, but any steps it would have taken toward the blocked path are considered avoided steps. The number of avoided steps is calculated as: avoided steps = total steps planned − current step taken
  - Number of objects collected

INPUT FOR NEWLY ADDED FUNCTION

`current_step` (integer) : The current step number of the robot in the simulation. This is used to calculate the number of avoided steps.

OUTPUT FOR NEWLY ADDED FUNCTION

Returns a Boolean value:

1. `True` → The robot successfully moved to the target position.
2. `False` → The robot could not move (e.g., all nearby positions blocked).

Updates:

1. The robot's position (self.x, self.y) is updated if movement occurs.
2. self.nearest_path_selected is incremented by 1 each time the method selects a nearest path.
3. self.path_avoided is updated as (1000 - current_step), representing the steps avoided by choosing the alternative path.
4. The robot may also collect a bio-hazard object if it moves to a position containing one (via the self.perceive() method called within self.move()).

Overall output:

```
Simulation 100

Starting robot simulation...
Initial position: (10, 79)
Move history:
DOWN -> LEFT -> UP -> UP -> UP -> LEFT -> DOWN -> LEFT -> LEFT -> DOWN -> RIGHT -> DOWN -> RIGHT -> DOWN -> RIGHT -> DOWN -> RIGHT ->
RIGHT -> UP -> LEFT -> RIGHT -> RIGHT -> DOWN -> DOWN -> DOWN -> RIGHT -> UP -> UP -> RIGHT -> RIGHT -> RIGHT -> RIGHT -> RIGHT -> UP
-> LEFT -> LEFT -> LEFT -> LEFT -> UP -> UP -> UP -> UP -> UP -> UP -> RIGHT -> UP -> RIGHT -> DOWN -> DOWN -> RIGHT -> UP -> UP -> RI
GHT -> UP -> RIGHT -> DOWN -> DOWN -> DOWN -> LEFT -> DOWN -> RIGHT -> RIGHT -> UP -> UP -> UP -> UP -> UP -> LEFT -> UP -> LEFT -> UP
 -> RIGHT -> UP -> RIGHT -> DOWN -> RIGHT -> UP -> UP -> RIGHT -> UP -> UP -> RIGHT -> RIGHT -> UP -> RIGHT -> RIGHT -> RIGHT -> DOWN
-> DOWN -> DOWN -> RIGHT -> RIGHT -> RIGHT -> RIGHT -> UP -> UP -> UP -> UP -> LEFT -> UP -> UP -> LEFT -> UP -> RIGHT -> RIGHT -> DOW
N -> DOWN -> RIGHT -> DOWN -> DOWN -> RIGHT -> DOWN -> LEFT -> DOWN -> RIGHT -> RIGHT -> UP -> UP -> RIGHT -> RIGHT -> DOWN -> DOWN ->
 LEFT -> DOWN -> LEFT -> LEFT -> DOWN -> RIGHT -> RIGHT -> RIGHT -> RIGHT -> RIGHT -> RIGHT -> RIGHT -> DOWN -> DOWN -> RIGHT -> RIGHT
 -> RIGHT -> RIGHT -> UP -> LEFT -> LEFT -> UP -> LEFT -> UP -> LEFT -> UP -> LEFT -> UP -> LEFT -> LEFT -> UP -> RIGHT -> RIGHT -> RIGHT -> DOWN ->
 RIGHT -> UP -> RIGHT -> UP -> RIGHT -> UP -> LEFT -> DOWN -> LEFT -> LEFT -> LEFT -> UP -> LEFT -> UP -> LEFT -> LEFT -> LEFT -> DOWN
 -> LEFT -> LEFT -> UP -> RIGHT -> UP -> RIGHT -> RIGHT -> UP -> UP -> RIGHT -> UP -> RIGHT -> UP -> RIGHT -> UP -> RIGHT -> DOWN -> LEFT ->
 DOWN -> DOWN -> RIGHT -> RIGHT -> UP -> LEFT -> DOWN -> DOWN -> LEFT -> LEFT -> UP -> LEFT -> DOWN -> LEFT -> RIGHT -> DOWN -> DOWN -
> RIGHT -> DOWN -> LEFT -> LEFT -> DOWN -> DOWN -> DOWN -> RIGHT -> RIGHT -> DOWN -> DOWN -> LEFT -> DOWN -> DOWN -> DOWN -> RIGHT ->
RIGHT -> DOWN -> DOWN -> DOWN -> LEFT -> UP -> UP -> LEFT -> DOWN -> RIGHT -> DOWN -> DOWN -> DOWN -> RIGHT -> RIGHT -> RIGHT -> UP ->
 RIGHT -> UP -> LEFT -> UP -> LEFT -> DOWN -> LEFT -> DOWN -> UP -> UP -> UP -> RIGHT -> UP -> UP -> RIGHT -> DOWN -> RIGHT -> RIGHT -
> UP -> RIGHT -> RIGHT -> DOWN -> RIGHT -> RIGHT -> DOWN -> DOWN -> RIGHT -> UP -> UP -> UP -> DOWN -> DOWN -> RIGHT -> RIGHT -> UP ->
 UP -> LEFT -> UP -> LEFT -> UP -> RIGHT -> RIGHT -> DOWN -> RIGHT -> UP -> RIGHT -> RIGHT -> DOWN -> LEFT -> DOWN -> RIGHT -> RIGHT -
> DOWN -> RIGHT -> UP -> UP -> UP -> LEFT -> LEFT -> LEFT -> RIGHT -> LEFT -> LEFT -> DOWN -> DOWN -> DOWN -> RIGHT -> DOWN -> RIGHT -
> UP -> LEFT -> LEFT -> DOWN -> DOWN -> LEFT -> LEFT -> DOWN -> LEFT -> DOWN -> DOWN -> LEFT -> DOWN -> LEFT -> LEFT -> LEFT -> UP ->
UP -> RIGHT -> UP -> LEFT -> LEFT -> DOWN -> RIGHT -> RIGHT -> DOWN -> RIGHT -> UP -> RIGHT -> UP -> DOWN -> RIGHT -> RIGHT -> RIGHT -
> RIGHT -> UP -> LEFT -> RIGHT -> RIGHT -> DOWN -> RIGHT -> RIGHT -> UP -> LEFT -> UP -> LEFT -> DOWN -> RIGHT -> DOWN -> DOWN -> LEFT
 -> DOWN -> LEFT -> LEFT -> UP -> RIGHT -> LEFT -> LEFT -> DOWN -> DOWN -> LEFT -> UP -> LEFT -> UP -> LEFT -> LEFT -> LEFT -> DOWN ->
 DOWN -> LEFT -> DOWN -> DOWN -> RIGHT -> DOWN -> RIGHT -> RIGHT -> RIGHT -> UP -> RIGHT -> UP -> LEFT -> LEFT -> UP -> LEFT -> DOWN -
> DOWN -> RIGHT -> LEFT -> UP -> LEFT -> RIGHT -> RIGHT -> LEFT -> RIGHT -> RIGHT -> UP -> UP -> LEFT -> RIGHT -> DOWN -> UP -> RIGHT
-> UP -> LEFT -> UP -> DOWN -> RIGHT -> UP -> RIGHT -> RIGHT -> UP -> RIGHT -> DOWN -> LEFT -> DOWN -> UP -> DOWN -> UP -> LEFT -> RIG
HT -> LEFT -> RIGHT -> LEFT -> LEFT -> RIGHT -> RIGHT -> RIGHT -> DOWN -> RIGHT -> RIGHT -> RIGHT -> DOWN -> RIGHT -> UP -> UP -> UP -
> RIGHT -> RIGHT -> UP -> RIGHT -> RIGHT -> DOWN -> RIGHT -> RIGHT -> RIGHT -> RIGHT -> DOWN -> RIGHT -> DOWN -> LEFT -> LEFT -> LEFT
-> DOWN -> DOWN -> RIGHT -> RIGHT -> RIGHT -> DOWN -> RIGHT -> UP -> UP -> LEFT -> LEFT -> LEFT -> UP -> UP -> LEFT -> LEFT -> DOWN ->
 DOWN -> DOWN -> DOWN -> LEFT -> DOWN -> LEFT -> UP -> UP -> LEFT -> LEFT -> LEFT -> DOWN -> RIGHT -> DOWN -> LEFT -> LEFT -> UP -> UP
 -> LEFT -> LEFT -> LEFT -> LEFT -> DOWN -> DOWN -> LEFT -> UP -> UP -> UP -> LEFT -> DOWN -> LEFT -> LEFT -> DOWN -> LEFT -> DOWN ->
RIGHT -> LEFT -> LEFT -> UP -> DOWN -> UP -> DOWN -> RIGHT -> DOWN -> DOWN -> LEFT -> LEFT -> DOWN -> DOWN -> LEFT -> DOWN -> LEFT ->
UP -> UP -> UP -> UP -> UP -> RIGHT -> DOWN -> DOWN -> DOWN -> RIGHT -> RIGHT -> DOWN -> DOWN -> LEFT -> DOWN -> LEFT -> LEFT -> LEFT
-> LEFT -> LEFT -> LEFT -> UP -> LEFT -> LEFT -> LEFT -> UP -> UP -> RIGHT -> UP -> UP -> RIGHT -> DOWN -> DOWN -> RIGHT -> RIGHT -> D
OWN -> RIGHT -> UP -> UP -> LEFT -> UP -> RIGHT -> RIGHT -> UP -> UP -> LEFT -> LEFT -> LEFT -> LEFT -> UP -> RIGHT -> RIGHT -> RIGHT
-> UP -> RIGHT -> RIGHT -> RIGHT -> UP -> DOWN -> DOWN -> DOWN -> LEFT -> UP -> RIGHT -> LEFT -> RIGHT -> DOWN -> DOWN -> LEFT -> UP -
> UP -> UP -> UP -> DOWN -> RIGHT -> UP -> LEFT -> LEFT -> UP -> RIGHT -> RIGHT -> RIGHT -> DOWN -> LEFT -> LEFT -> UP -> LEFT -> UP -
> DOWN -> RIGHT -> RIGHT -> LEFT -> LEFT -> RIGHT -> RIGHT -> RIGHT -> RIGHT -> UP -> UP -> UP -> RIGHT -> DOWN -> RIGHT -> UP -> UP -
> UP -> UP -> RIGHT -> UP -> LEFT -> LEFT -> DOWN -> LEFT -> UP -> LEFT -> UP -> UP -> LEFT -> UP -> RIGHT -> UP -> UP -> DOWN -> DOWN
 -> DOWN -> DOWN -> LEFT -> LEFT -> UP -> DOWN -> LEFT -> DOWN -> UP -> RIGHT -> DOWN -> DOWN -> UP -> UP -> LEFT -> UP -> LEFT -> RIG
HT -> LEFT -> RIGHT -> LEFT -> RIGHT -> UP -> RIGHT -> DOWN -> DOWN -> DOWN -> UP -> DOWN -> UP -> UP -> LEFT -> UP -> RIGHT -> DOWN -
> DOWN -> UP -> RIGHT -> DOWN -> RIGHT -> LEFT -> LEFT -> LEFT -> LEFT -> RIGHT -> RIGHT -> LEFT -> RIGHT -> UP -> LEFT -> LEF
T -> DOWN -> UP -> UP -> DOWN -> DOWN -> LEFT -> UP -> LEFT -> LEFT -> LEFT -> UP -> DOWN -> UP -> DOWN -> UP -> UP -> RIGHT -> DOWN -
> LEFT -> LEFT -> LEFT -> UP -> RIGHT -> DOWN -> DOWN -> UP -> DOWN -> UP -> LEFT -> UP -> UP -> DOWN -> RIGHT -> LEFT -> LEFT -> UP -
> LEFT -> LEFT -> LEFT -> UP -> LEFT -> UP -> LEFT -> UP -> RIGHT -> UP -> UP -> RIGHT -> UP -> RIGHT -> RIGHT -> RIGHT -> DOWN -> RIG
HT -> RIGHT -> DOWN -> LEFT -> LEFT -> DOWN -> RIGHT -> RIGHT -> DOWN -> LEFT -> DOWN -> UP -> RIGHT -> DOWN -> LEFT -> DOWN -> LEFT -
> RIGHT -> DOWN -> UP -> DOWN -> LEFT -> UP -> LEFT -> LEFT -> LEFT -> DOWN -> LEFT -> LEFT -> LEFT -> LEFT -> DOWN -> RIGHT -> DOWN -
> DOWN -> DOWN -> LEFT -> LEFT -> UP -> LEFT -> UP -> UP -> DOWN -> RIGHT -> UP -> LEFT -> RIGHT -> LEFT -> LEFT -> LEFT -> LEFT -> UP
```

```
DOWN
---- Robot Performance ----
Collected objects: 406
Total moves: 1000
Total actions:  1406
Efficiency (objects per move): 0.406
Collected objects: [(8, 77), (7, 78), (6, 78), (7, 80), (8, 81), (9, 82), (10, 82), (11, 82), (11, 81), (10, 81), (12, 81), (12, 83),
(13, 83), (13, 82), (14, 82), (15, 82), (17, 82), (18, 81), (15, 81), (14, 81), (14, 80), (14, 79), (14, 78), (14, 76), (14, 75), (15,
 75), (15, 74), (16, 75), (16, 76), (17, 76), (17, 75), (17, 74), (18, 74), (18, 73), (19, 73), (19, 75), (19, 76), (18, 77), (19, 77)
, (20, 76), (20, 73), (20, 72), (19, 70), (19, 69), (20, 69), (20, 70), (21, 70), (22, 67), (22, 66), (23, 66), (24, 66), (24, 65), (2
7, 65), (27, 66), (28, 68), (29, 68), (30, 68), (30, 64), (30, 62), (29, 62), (29, 61), (30, 61), (31, 61), (31, 62), (32, 63), (32, 6
4), (33, 66), (32, 66), (34, 65), (36, 65), (36, 66), (35, 68), (33, 68), (35, 69), (37, 69), (38, 69), (39, 69), (40, 70), (40, 71),
(41, 71), (42, 71), (44, 71), (42, 70), (41, 68), (39, 67), (38, 67), (38, 66), (41, 66), (42, 67), (42, 66), (43, 66), (43, 65), (42,
 65), (41, 65), (40, 64), (39, 64), (38, 63), (37, 63), (36, 63), (36, 64), (35, 64), (34, 64), (34, 63), (36, 62), (37, 61), (37, 60)
, (38, 60), (39, 58), (40, 58), (41, 58), (41, 59), (40, 59), (40, 60), (41, 61), (42, 60), (41, 60), (39, 62), (39, 61), (38, 61), (3
8, 64), (39, 65), (38, 68), (39, 68), (39, 70), (38, 71), (38, 73), (39, 73), (40, 74), (40, 75), (40, 76), (39, 75), (39, 74), (40, 7
8), (41, 78), (42, 78), (42, 77), (42, 75), (41, 75), (40, 77), (41, 74), (41, 72), (42, 72), (42, 73), (43, 73), (44, 73), (44, 72),
(45, 72), (46, 72), (46, 73), (47, 73), (48, 73), (48, 75), (49, 75), (49, 74), (50, 74), (51, 73), (51, 72), (50, 71), (49, 70), (54,
 70), (54, 71), (53, 71), (54, 72), (55, 73), (56, 73), (56, 72), (56, 71), (56, 70), (52, 72), (53, 73), (53, 74), (54, 74), (54, 73)
, (52, 74), (52, 75), (51, 75), (50, 75), (50, 76), (49, 76), (48, 78), (48, 79), (46, 79), (45, 79), (45, 78), (46, 77), (46, 76), (4
4, 76), (46, 78), (47, 78), (47, 77), (48, 77), (48, 76), (51, 77), (52, 76), (51, 76), (53, 77), (54, 77), (55, 77), (55, 76), (54, 7
6), (54, 75), (53, 75), (54, 78), (52, 79), (51, 78), (52, 78), (50, 78), (50, 80), (49, 80), (49, 79), (44, 80), (44, 81), (45, 82),
(46, 83), (48, 83), (49, 82), (48, 81), (47, 81), (47, 80), (46, 80), (46, 81), (46, 82), (47, 82), (45, 81), (55, 78), (55, 79), (56,
 79), (56, 78), (56, 77), (56, 76), (57, 76), (60, 75), (63, 76), (64, 77), (65, 78), (64, 78), (62, 79), (62, 80), (64, 80), (65, 80)
, (66, 81), (66, 80), (66, 79), (64, 79), (63, 79), (63, 77), (61, 80), (59, 82), (59, 81), (58, 80), (57, 80), (56, 80), (56, 81), (5
7, 81), (57, 82), (56, 82), (55, 81), (55, 80), (53, 80), (52, 80), (51, 82), (50, 81), (46, 84), (45, 84), (43, 86), (42, 87), (42, 8
3), (45, 85), (45, 86), (44, 87), (43, 88), (38, 88), (38, 87), (37, 87), (35, 86), (35, 85), (36, 85), (36, 83), (37, 83), (37, 84),
(38, 85), (40, 85), (39, 84), (41, 83), (41, 82), (40, 81), (39, 81), (38, 81), (39, 80), (40, 79), (41, 79), (42, 79), (43, 79), (43,
 78), (43, 81), (42, 80), (45, 75), (45, 74), (47, 74), (48, 71), (47, 70), (46, 70), (46, 71), (45, 71), (45, 70), (44, 68), (43, 67)
, (44, 67), (44, 66), (42, 68), (41, 70), (36, 68), (30, 65), (29, 65), (27, 63), (27, 62), (28, 62), (28, 60), (29, 59), (30, 59), (3
1, 59), (32, 59), (33, 60), (34, 60), (33, 61), (32, 61), (33, 63), (28, 66), (26, 66), (25, 66), (26, 67), (26, 69), (26, 70), (25, 7
0), (24, 69), (23, 69), (23, 68), (23, 67), (24, 68), (21, 67), (20, 67), (20, 66), (19, 67), (20, 68), (18, 68), (18, 67), (17, 67),
(17, 66), (18, 66), (17, 65), (16, 65), (15, 66), (17, 68), (16, 68), (15, 68), (15, 69), (14, 69), (14, 68), (14, 67), (14, 66), (12,
 66), (12, 67), (12, 68), (11, 68), (11, 69), (10, 67), (8, 66), (7, 67), (6, 67), (6, 68), (4, 68), (3, 70), (2, 70), (1, 70), (3, 71
), (4, 71), (5, 71), (6, 69), (6, 70), (8, 72), (8, 71), (8, 70), (9, 69), (9, 70), (10, 70), (11, 70), (15, 71), (15, 73), (13, 73),
(12, 73), (12, 72), (12, 71), (13, 71), (18, 72), (22, 71), (23, 71), (22, 72), (22, 73), (23, 73), (23, 74), (21, 73), (21, 72), (17,
 71), (16, 71), (16, 70), (15, 78)]
Final position: (15, 78)
Accessibility checks: 3695
BioHazard checks: 1001
Total percepts: 4696
Movement actions: 1000
Cleaning actions: 406
Number of human encountered: 146
Number of nearest path selected : 146
Avoided:  58797
Number of object collected: 406
PS D:\3-2\Artificial Intelligence\Lab>
```

Example Behavior:

- Input: current_step = 200
- Robot is at (5, 5) and encounters a human.
- Nearest bio-hazard at (5, 6) → Robot moves "DOWN".
- Output: True
- Updates: self.nearest_path_selected += 1, self.path_avoided += 800

DATA STRUCTURE

Data structures used in this task are:

1. **Tuple `(x, y)`:** This represents a single coordinate on the grid. Tuples are used for robot positions, human positions, bio-hazard locations, and individual cells of non-accessible areas. Their immutability allows them to be safely stored in sets and lists.
2. **List[x, y, z…]**: Lists are used to store ordered collections, such as the robot's move history, collected objects, human positions, bio-hazard locations, and non-accessible areas. Lists provide easy iteration and maintain the order of insertion.
3. **Set**{x, y, z..}: The robot's visited positions are stored in a set to ensure uniqueness and allow fast membership checks. This prevents the robot from revisiting the same cell unnecessarily.

4. **Dictionary**: The `non_accessible_info` dictionary maps the top-left corner of a non-accessible area to its height and width. This allows easy lookup of area dimensions when generating blocked regions in the grid.
5. **2D List (Matrix)**: The environment grid is represented as a 2D list where each cell can be accessible, non-accessible, a bio-hazard, or a human. The 2D list allows direct access using row and column indices and efficiently represents the entire environment.

ALGORITHM

ALGORITHM:

Start

1. Identify the four neighboring positions around the robot: right, left, down, and up.
2. Check each neighboring position: If a bio-hazard is present, mark it as the target position.
3. If no bio-hazard is found:
    - Randomly shuffle the neighboring positions.
    - Select the first accessible position (even if previously visited) as the target.

4. Determine the movement direction to the target:
    - Right if the target's x-coordinate is greater.
    - Left if smaller.
    - Down if the target's y-coordinate is greater.
    - Up if smaller.
5. Update robot statistics:
    - Increment nearest_path_selected by 1.
    - Add (1000 - current_step) to path_avoided.

6. Move the robot in the chosen direction.
7. If the robot moves to a bio-hazard, collect it automatically.
8. Return True if the move was successful; otherwise, return False.

End

PSEUDOCODE:

```
Function move_to_nearest_object(current_step):

    Initialize target = None
    Initialize near_positions as empty list
    Add (x+1, y) to near_positions
    Add (x-1, y) to near_positions
    Add (x, y+1) to near_positions
    Add (x, y-1) to near_positions
```

```
For each (near_x, near_y) in near_positions:
        If (near_x, near_y) is inside grid AND has bio-hazard:
            target = (near_x, near_y)
If target is None:
        Shuffle near_positions randomly
        For each (nx, ny) in near_positions:
            If (nx, ny) is inside grid AND can move there (allow visited):
                target = (nx, ny)
                Break
    tar_x, tar_y = target
    If tar_x > current x: direction = "RIGHT"
    Else if tar_x < current x: direction = "LEFT"
    Else if tar_y > current y: direction = "DOWN"
    Else: direction = "UP"
    Increment nearest_path_selected by 1
    Increment path_avoided by (1000 - current_step)
    Return move(direction)
```

TEST CASE

Test Case for test_move_to_nearest_object_behavior():

Test Case ID	Test Function Name	Test Objective	Test Steps & Input Data	Expected Result	Status
TC-01	test_move_to_nearest_object_behavior()	Verify robot moves to nearest bio-hazard when human encountered or accessible cell and updates statistics correctly	1. Create environment of size 5×5 with 0 bio-hazards. 2. Place a bio-hazard at (2,2) and update env.bio_hazard. 3. Initialize robot at (2,1) with visited={(2,1)}, nearest_path_selected=0, path_avoided=0. 4. Set current_step=1 and call robot.move_to_nearest_object(current_step). 5. Re-initialize environment of size 5×5 with 0 bio-hazards. 6. Place robot at (2,2) with visited={(2,2)}, nearest_path_selected=0, path_avoided=0. 7. Call robot.move_to_nearest_object(current_step) again.	First move: - Robot position = (2,2) - nearest_path_selected = 1 - path_avoided = 999 - (2,2) is in collected_objects Second move: - Robot moves to one of (3,2), (1,2), (2,3), (2,1) - nearest_path_selected = 1 - path_avoided = 999	Pass